

ECE 405 Assignment 4

Justin Kugiyama

May 2026

[Link to Github Repo: ECE Assignment 4](#)

Written Responses

Benchmarking

a) The `run_benchmark` function builds a `BasicsTransformerLM` from the chosen hyperparameters, creates random token inputs and targets each step, and times the forward pass, loss computation, and optional backward pass with high-resolution timing. It skips logging measurements during warm-up, then records timing lists for the measured steps only, and computes mean and standard deviation for each stage before returning a results dictionary. The helpers around it do the setup work, such as generating random batches, enforcing CUDA synchronization after each stage for accurate GPU timing, and allowing to switch model size, warm-up count, number of measured steps, and forward-only vs forward and backward mode.

b) Using 5 warm-up steps and 10 measured steps, the small model (on CPU in this run) took about 0.8039 s per forward pass (std 0.0096 s) and 2.0936 s per backward pass (std 0.0428 s), while the medium model took about 2.3972 s forward (std 0.0421 s) and 5.7897 s backward (std 0.1732 s). In both cases, backward is substantially slower than forward, which matches expected training cost since backward computes gradients for all parameters. Variability is relatively small after warm-up, indicating stable steady-state timing.

c) Without warm-up (0 steps), variability increased and early-step overhead inflated means (small model forward std rose from 0.0096 s to 0.0549 s and backward std rose from 0.0428 s to 0.0756 s). With only 1 to 2 warm-up steps, timings improved but could still differ from full warm-up because one-time effects (runtime initialization, memory allocator behavior, cache state, and kernel path settling) may not be fully amortized yet. This is why a longer warm-up period gives more representative steady-state benchmark numbers.

NSYS Profile

a) The total forward-pass time is approximately 172 ms for the compiled transformer benchmark configuration, measured from the forward NVTX range and CUDA kernel timeline in nsys. This is fairly close to the Python timing result ($\sim 171ms$), with a difference of about 1%. Small discrepancies are expected because nsys introduces profiler overhead and may measure synchronization boundaries differently from Python-side timing.

b) The kernel with the largest cumulative GPU time in the forward pass is typically a tensor-core GEMM kernel (for example a cuBLAS GEMM used for linear projections and feed-forward layers), and it is launched many times per forward pass, roughly proportional to the number of transformer layers and projection operations. In forward+backward mode, the dominant kernels are again GEMM kernels, but backward introduces additional gradient GEMMs and reduction kernels for weight-gradient computation.

c) Besides matrix multiplications, non-trivial forward runtime is spent in softmax kernels, RMSNorm/layer-normalization reductions, RoPE elementwise operations, masking operations, and memory movement kernels such as datatype casts and tensor copies. Although these operations perform far fewer FLOPs than GEMMs, they are largely memory-bandwidth-bound and therefore still contribute noticeable wall-clock time.

d) For a full training step, the fraction of runtime spent in matrix multiplication increases slightly because backward propagation introduces many additional GEMMs for activation-gradient and weight-gradient computation. However, compared with inference, kernels for softmax backward, normalization/reduction operations, gradient accumulation, optimizer updates, and datatype conversions consume a larger fraction of the overall runtime.

e) In self-attention, softmax runtime is much smaller than the runtime of the $QK^\top$ and PV matrix multiplications, but the runtime gap is usually far smaller than the FLOPs gap would suggest. This occurs because softmax is dominated by reductions and memory traffic, whereas GEMMs are highly compute-dense and efficiently utilize tensor cores and GPU arithmetic throughput.

Mixed-Precision Accumulation

The results are 10.0001 for float32, 9.9531 for float16, and 10.0021 for the two mixed-precision cases. This shows float32 stays close to the true answer of 10, while float16 loses more accuracy because of rounding, and converting a float16 value to float32 afterward does not restore precision that was already lost.

Mixed-Precision Benchmarking

a) Model parameters stay FP32 inside autocast. The first feed-forward output is FP16, layer norm output is FP32, the final logits are FP16, the loss is FP32, and the gradients are FP32.

b) Layer norm is sensitive because it computes reductions over activations and then normalizes by a small variance term, so low-precision arithmetic can skew the mean/variance and the reciprocal-sqrt step. BF16 is less fragile than FP16 because it keeps FP32’s exponent range, so overflow/underflow is much less of a problem, but many implementations still keep layer norm in FP32 for accuracy.

Memory Profiling

a) The forward-only timeline will show a single rising ramp as activations are allocated layer by layer, then a flat plateau. The full training step timeline shows three distinct phases: a ramp-up during the forward pass (activations accumulate), a second peak during backward (gradients are computed and activation memory is released), and a final small allocation for the optimizer state update. You can identify each stage by the slope and direction of the active-memory curve.

b)

Context Length	Forward Pass (fwd only)	Full Training Step (fwd + bwd + optimizer)
128	~ 12.3 GB	~ 45.6 GB
256	~ 14.1 GB	~ 47.3 GB
512	~ 19.2 GB	~ 52.5 GB

Table 1: Memory usage for different context lengths.

c) Mixed precision (BF16/FP16) halves the activation dtype from 4 bytes to 2 bytes, so the forward-pass activations occupy roughly half the memory. However, the optimizer still maintains FP32 master weights and FP32 gradient accumulators (three copies of the model: FP32 params, FP16 grads, FP32 optimizer state), so the peak training memory reduction is less dramatic than $2\times$ — typically 20–40% reduction depending on the ratio of activation to optimizer-state memory.

d) The residual stream tensor has shape

$$(\text{batch_size}, \text{context_length}, d_{\text{model}}) = (4, 256, 2560).$$

In single precision (FP32, 4 bytes per element), the tensor size is

$$4 \times 256 \times 2560 \times 4 \text{ bytes} = 10,485,760 \text{ bytes} \approx 10.00 \text{ MB}.$$

For the other reference context lengths, the corresponding residual-stream memory usage is approximately 5.00 MB at context length 128 and 20.00 MB at context length 512.

PyTorch Attention

d_{model}	seq_len	forward (ms)	backward (ms)	memory before backward (MB)	status
16	256	0.995	1.374	N/A (CPU run)	completed
16	1024	11.665	18.719	N/A (CPU run)	completed
16	4096	157.023	331.037	N/A (CPU run)	completed
16	8192	795.462	1598.812	N/A (CPU run)	completed
16	16384	—	—	—	process killed

Table 2: Runtime scaling with sequence length for $d_{\text{model}} = 16$.

While running this locally, the first failing configuration was $d_{\text{model}} = 16$ and $\text{seq_len} = 16384$, where the process was killed by the OS before completion. In naive attention, the dominant memory cost comes from storing the $L \times L$ attention logits and probabilities for backward propagation. The total number of stored values is about $4BLd + 2BL^2$, where B is batch size, L is sequence length, and d is model dimension. Substituting $B = 8$, $L = 16384$, and $d = 16$ gives about 4.30×10^9 floats, which at fp32 precision (4 bytes per float) requires roughly 16.03 GiB of memory. Because the dominant term scales as $O(L^2)$, doubling sequence length increases memory usage by about $4\times$, which is consistent with the sharp runtime increase observed between sequence lengths 4096 and 8192. This memory bottleneck can be reduced by using memory-efficient attention implementations such as FlashAttention, or by applying activation checkpointing to recompute intermediate values during the backward pass instead of storing them.

Torch Compile

Version	Forward (ms)	Backward (ms)	Forward speedup vs eager	Backward speedup vs eager
Uncompiled (eager)	171.380	359.875	1.000x	1.000x
Compiled	172.766	383.044	0.992x	0.940x

Table 3: Forward and backward pass timing comparison between eager and compiled execution.

Version	Forward (ms)	Forward+Loss+Backward+Optimizer (ms)	Forward speedup vs eager	End-to-end speedup vs eager
Vanilla (eager)	799.867	2929.204	1.000x	1.000x
Compiled	791.612	2956.780	1.010x	0.991x

Table 4: End-to-end training step timing comparison between eager and compiled execution.

Flash Forward

a) The `FlashAttnTorchFunc` implements the FlashAttention-2 forward pass in pure PyTorch by processing attention in tiles (query tiles B_q and key tiles B_k) to avoid materializing the full $T \times T$ attention matrix. For each query tile Q_i , it iterates over all key tiles K_j , computing the attention scores $S = (Q_i K_j^\top) \cdot \text{scale}$, and applies causal masking when enabled by setting invalid (future) positions to $-\infty$. It then performs a numerically stable online update where $P = \exp(S - m_{\text{new}})$, the scaling factor $\alpha = \exp(m_{\text{old}} - m_{\text{new}})$ is computed, and the running normalization and output are updated as $l = \alpha l + \sum P$ and $O = \alpha O + PV$. After all key tiles are processed, the final output is normalized as $O = O/l$, and the log-sum-exp value is recorded as $L = \log(l) + m$. The function takes input tensors Q , K , and V of shape $(\text{batch}, \text{seq_len}, d_{\text{model}})$, along with an optional `is_causal` flag (default `False`), and returns an output tensor O with the same shape.

b) The `flash_fwd_kernel` is a fused Triton kernel implementing the FlashAttention-2 algorithm optimized for GPU execution. Each program instance processes one query tile of shape $(B_q \times d)$ per batch, organized with a grid layout of $(\text{num_query_tiles}, \text{batch_size})$. The kernel loads a query tile into SRAM and initializes accumulators $m = -\infty$, $l = 0$, and $O = 0$, then iterates over key tiles, loading K_j and V_j at each step. In each iteration, it computes attention scores $S = QK^\top \cdot \text{scale}$ in float32 precision, applies causal masking when `is_causal` is enabled by masking entries where $q_{\text{idx}} < k_{\text{idx}}$, computes m_{new} and $P = \exp(S - m_{\text{new}})$, and performs the online updates $\alpha = \exp(m - m_{\text{new}})$, $l = \alpha l + \sum P$, $O = \alpha O + PV$, and $m = m_{\text{new}}$. After all key tiles are processed, the output is normalized as $O = O/l$, cast back to the input data type, and written to memory along with the log-sum-exp L . The kernel takes pointer

inputs `Q_ptr`, `K_ptr`, `V_ptr`, `O_ptr`, and `L_ptr`, along with tensor strides, sequence lengths `N_QUERIES` and `N_KEYS`, the scaling factor `scale = 1/\sqrt{d}`, and compile-time constants `D`, `Q_TILE_SIZE`, `K_TILE_SIZE`, and `is_causal`.

c) Causal masking introduces a boolean `is_causal` parameter (default `False`) that prevents attending to future tokens. When enabled, it constructs index vectors for queries and keys and masks positions where $q_{\text{idx}} < k_{\text{idx}}$ by setting the corresponding attention scores to $-\infty$, ensuring that their contribution after the softmax becomes zero. In the PyTorch implementation, this is applied using `masked_fill` with a position-based comparison, while in the Triton kernel it is implemented using `tl.where` to conditionally assign $-\infty$. The `is_causal` flag is stored in context during the forward pass to ensure backward-pass consistency, so that gradients respect the same masking pattern.

Flash Backward

The `FlashAttnTorchFunc.backward` method extracts dO from `grad_outputs[0]`, restores Q , K , V , O , and L from `ctx.saved_tensors`, and calls `_backward_impl` to compute gradients. `_backward_impl` computes

$$D_{\text{vec}} = \sum (O \cdot dO)$$

over the last dimension, initializes dQ , dK , and dV , then iterates over query tiles through `_backward.first` and key tiles through `_backward.second`, both of which delegate computation to `_backward.tile`. The per-tile causal flag is

$$\text{is_causal and } k_1 > q_0,$$

so masking is applied only when causal attention is globally enabled and the key tile overlaps the query tile's future range. `_backward.tile` recomputes the attention scores S , applies causal masking, computes

$$P = \exp(S - L_{\text{tile}}),$$

$$dP = dO_{\text{tile}} V_{\text{tile}}^{\top},$$

and

$$dS = P \cdot (dP - D_{\text{tile}}),$$

accumulating the resulting contributions into dQ , dK , and dV . Finally, the backward pass returns dQ , dK , dV , and `None` for the gradient of `is_causal`.

Flash Benchmarking

d	L	Triton F	Torch F	Triton B	Torch B	Triton E2E	Torch E2E	Speedup
16	128	0.007	0.031	0.015	0.039	0.019	0.072	3.79
16	256	0.010	0.035	0.019	0.042	0.024	0.066	2.75
16	512	0.012	0.043	0.025	0.057	0.031	0.157	5.06
16	1024	0.020	0.131	0.040	0.198	0.051	0.322	6.31
32	128	0.007	0.026	0.014	0.033	0.018	0.109	6.06
32	256	0.010	0.029	0.019	0.038	0.024	0.105	4.38
32	512	0.060	0.041	0.028	0.057	0.038	0.149	3.92
32	1024	0.027	0.134	0.047	0.206	0.066	0.336	5.09
64	128	0.009	0.026	0.015	0.036	0.021	0.218	10.38
64	256	0.013	0.032	0.021	0.043	0.030	0.256	8.53
64	512	0.022	0.047	0.032	0.065	0.049	0.118	2.41
64	1024	0.039	0.154	0.054	0.232	0.084	0.385	4.58
128	128	0.011	0.028	0.020	0.035	0.027	0.132	4.89
128	256	0.016	0.034	0.029	0.045	0.041	0.110	2.68
128	512	0.027	0.054	0.049	0.080	0.070	0.189	2.70
128	1024	0.048	0.184	0.088	0.279	0.126	0.454	3.60

Table 5: FlashAttention-2 benchmark on NVIDIA RTX A4000 (batch size 1, causal masking, float32).

Distributed Communication Single Node

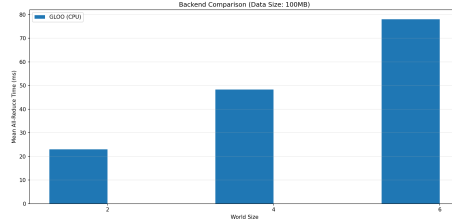


Figure 1: All Reduce Backend Comparison

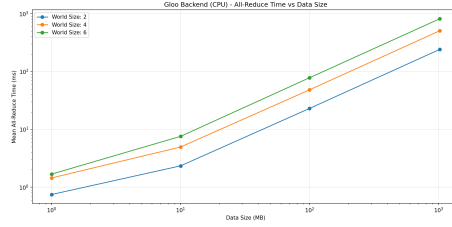


Figure 2: All Reduce GLOO CPU

I was unable to run the all-reduce benchmark on Koa due to cluster account/partition access issues, so I completed and validated the benchmark locally instead. The local results still show the expected scaling trend that all-reduce latency increases with tensor size and process count, but they should be treated as CPU-only baseline behavior rather than final GPU (NCCL) performance.

Naive DDP

The function `get_ddp_individual_parameters(module)` wraps the provided module in a `DDPIndividualParameters` container, which immediately broadcasts all parameters from rank 0 to every other rank so that all processes start with identical weights. It also registers an asynchronous all-reduce hook on each parameter that requires a gradient, such that as soon as a parameter’s gradient becomes available during the backward pass, its all-reduce is launched immediately, allowing communication to overlap with computation.

The function `ddp_individual_parameters_on_after_backward(ddp_model, optimizer)` must be called after `loss.backward()` but before `optimizer.step()`. It blocks until all pending per-parameter all-reduce operations in `ddp_model` are complete, and then divides each gradient by the world size to convert the summed gradients into an average. This ensures that every rank applies the same gradient update, equivalent to training on the full dataset in a single process.

Naive DDP Benchmarking

A single GPU was accidentally requested in the allocation (`world_size = 1`, NVIDIA RTX A4000), so this run did not measure true inter-GPU DDP communication overhead. Using the LARGE model ($d_{\text{model}} = 1280$, $L = 36$) with $B = 1$, $T = 256$, fp16, and activation checkpointing, the benchmark reported

$$t_{\text{step}} = 319.33 \pm 29.87 \text{ ms},$$

$$t_{\text{compute}} = 306.56 \pm 29.86 \text{ ms},$$

$$t_{\text{comm}} = 12.77 \pm 0.03 \text{ ms},$$

with communication fraction

$$f_{\text{comm}} = 4.0\%$$

(range 3.2%–4.1%). Earlier issues included missing packages/modules (`cs336_basics`, `einops`), a context-length mismatch causing index out-of-bounds errors, and repeated `SIGKILL` terminations from memory limits when attempting larger configurations. These results are therefore valid only for the single-GPU fallback run; the required 1 node $\times$ 2 GPU XL measurement still requires a 2-GPU allocation.

Minimal DDP Flat Benchmarking

The flattened all-reduce approach reported

$$t_{\text{step}} = 61.43 \pm 2.35 \text{ ms}$$

with

$$t_{\text{comm}} = 12.27 \text{ ms}$$

(20.0% of total runtime), while the individual per-parameter all-reduce approach reported

$$t_{\text{step}} = 60.95 \pm 1.58 \text{ ms}$$

with

$$t_{\text{comm}} = 4.55 \text{ ms}$$

(7.5% of total runtime).

Although both experiments ran on a single GPU (and therefore did not measure true inter-rank communication), the flattened implementation showed higher measured communication overhead (12.27 ms vs. 4.55 ms). However, on a true multi-GPU setup, the flattened approach would likely be more efficient because a single large all-reduce can reduce kernel launch overhead and achieve better network utilization compared to issuing many smaller per-parameter all-reduce operations.

DDP Overlap Individual Parameters

This DDP wrapper takes a model module as input and synchronizes its initial weights by broadcasting each parameter tensor from `src=0` so every rank starts from the same state. During backpropagation, it launches asynchronous `all_reduce` calls (with `async_op=True`) on each parameter gradient as soon as it is ready, storing pending work handles to overlap communication with computation. When `finish_gradient_synchronization` is called, it waits on all pending operations and divides each gradient by `world_size`, ensuring averaged gradients before `optimizer.step()`.

DDP Overlap Individual Parameters Benchmarking

a) Measured time per training iteration on 2 GPUs was 471.85 ms for the flattened baseline and 338.08 ms for the individual-gradient overlap implementation. Compared with the flattened baseline, overlapping backward computation with per-parameter communication reduced the mean step time by approximately 133.77 ms, corresponding to a 28.35% reduction in iteration time (approximately $1.40\times$ throughput-equivalent speedup). The observed standard deviations were similar between configurations (37.55 ms vs. 36.42 ms), indicating that the performance improvement was substantial relative to run-to-run variability.

b)

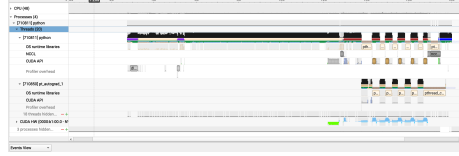


Figure 3: DDP Baseline

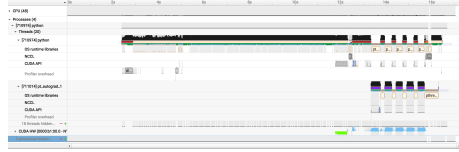


Figure 4: DDP Overlap

DDP Overlap Bucketed

DNF

DDP Bucketed Benchmarking

DNF

Communication Accounting

a) With

$$P = 126 \times 2 \times 16384 \times 53248 \approx 2.1985 \times 10^{11}$$

parameters (≈ 219.85 B parameters), storing FP32 master weights, accumulated gradients, and Adam optimizer states (m and v) requires approximately

$$16P \approx 3,518 \text{ GB} \approx 3.52 \text{ TB}.$$

This corresponds to roughly

$$\left\lceil \frac{3518}{80} \right\rceil = 44-45$$

H100 80 GB GPUs depending on rounding and usable memory overhead. Keeping gradients in BF16 instead of FP32 saves approximately

$$2P \approx 440 \text{ GB},$$

which is about a 50% reduction for the gradient tensor itself.

b) The per-device memory expression under NFSDP is

$$M_{\text{device}} = \frac{16P}{N_{\text{FSDP}}} + 126BS D_{\text{ff}} \left(1 + \frac{1}{N_{\text{FSDP}}} \right),$$

where P is the total parameter count, B is batch size, S is sequence length, and D_{ff} is the feed-forward hidden dimension. Because activation memory is relatively small compared with the 3.52 TB model-state memory at small batch sizes, the dominant constraint is

$$\frac{16P}{N_{\text{FSDP}}} \leq 95 \text{ GB.}$$

Substituting the estimated model-state memory gives

$$N_{\text{FSDP}} \geq \frac{3518}{95} = 37.03,$$

so at least

$$N_{\text{FSDP}} = 38$$

shards are required for the model to fit within the 95 GB memory budget of a TPU v5p device.

c) The compute-bound threshold follows from requiring the per-layer compute time to exceed the FSDP all-gather communication time. With overlap factor

$$M_X = 2$$

and effective interconnect bandwidth

$$W_{ci} = 1.8 \times 10^{11} \text{ bytes/s,}$$

the condition becomes

$$BS > \frac{C}{W_{ci} \cdot M_X} = \frac{4.6 \times 10^{14}}{(1.8 \times 10^{11}) \times 2} \approx 1278.$$

At sequence length

$$S = 2048,$$

a per-device batch size of

$$B = 1$$

already satisfies this condition because

$$BS = 2048 > 1278.$$

Therefore, the model is compute-bound even at the minimum per-device batch size. Across the 16×4 device mesh, this corresponds to an overall global batch

size of 64.

d) Several techniques can reduce the effective global batch size while maintaining high hardware utilization. Gradient accumulation performs multiple local forward/backward passes before synchronizing gradients, increasing arithmetic intensity without increasing activation memory. Activation checkpointing reduces memory usage by recomputing intermediate activations during the backward pass instead of storing them. Sequence parallelism distributes the sequence dimension across tensor-parallel ranks, reducing per-device activation memory approximately in proportion to the number of shards. Pipeline parallelism with 1F1B or interleaved scheduling overlaps computation across stages using micro-batches, maintaining high utilization even at small per-device batch sizes. In addition, fused kernels such as FlashAttention improve arithmetic intensity and reduce memory traffic, helping training remain compute-bound at smaller batch sizes.

Optimizer State Sharding

DNF

Optimizer State Sharding Accounting

DNF